

Custom SQL Queries

Unicorn Database • Written to support executive summary analysis • Federica Lucarelli & Sophia Sager

The following queries were written as custom extensions beyond the original 14 questions. Each one was designed to answer a specific analytical question raised during the project and to provide evidence directly supporting the claims made in the executive summary.

01 What is the total profit and profit margin for each product sub-category?

Result Tables: -\$17,733 at -8.6% margin | Bookcases: -\$3,479 at -3.0% | Supplies: -\$1,187 at -2.5%

```
SELECT
    product_subcategory,
    SUM(order_profits) AS total_profit,
    SUM(order_profits) / SUM(order_sales) AS profit_margin
FROM
    order_details JOIN product ON order_details.product_id = product.product_id
GROUP BY
    product_subcategory
ORDER BY
    total_profit ASC;
```

What it does: Groups all order lines by product sub-category and calculates total profit and profit margin for each one, ordered from least to most profitable. This reveals which sub-categories are loss-making across the full 2015 to 2018 period and which are generating healthy returns. Tables, Bookcases and Supplies are the three sub-categories producing a net loss.

02 What is the average profit and number of order lines at each discount level?

Result Profit is positive at 0%, 10% and 20%. At 30% average profit flips to a loss. Every level above 20% produces a loss.

```
SELECT
    od.order_discount,
    COUNT(*) AS num_lines,
    AVG(od.order_profits) AS avg_profit
FROM
    order_details od
GROUP BY
    od.order_discount
ORDER BY
    od.order_discount;
```

What it does: Groups every order line by its exact discount value and returns the average profit and line count at each level. This identifies the precise point at which discounting stops being a margin-preserving tool and starts generating losses. The result shows a consistent

and sharp transition at the 20% threshold, with every discount tier above it producing a negative average profit.

03 For each Furniture sub-category, how does profit differ between orders with no discount and orders where a discount was applied?

Result Tables at 0% discount: avg +\$184.35, total +\$13,273 across 72 lines. Tables with discount: avg -\$125.53, total -\$31,006 across 247 lines.

```
SELECT
    product_subcategory,
    CASE WHEN order_discount = 0 THEN 'No discount'
         ELSE 'Discount applied' END AS pricing,
    ROUND(AVG(order_profits)::numeric, 2) AS avg_profit,
    ROUND(SUM(order_profits)::numeric, 0) AS total_profit,
    COUNT(*) AS num_lines
FROM
    order_details
    JOIN product ON order_details.product_id = product.product_id
WHERE
    product_category = 'Furniture'
GROUP BY
    product_subcategory, pricing
ORDER BY
    product_subcategory, pricing;
```

What it does: Filters to Furniture orders only and splits each sub-category into two groups: those sold at full price and those sold at any discount. Comparing the average and total profit between the two groups isolates the effect of discounting from any other variable. The result proves that Furniture is profitable when sold at full price and loss-making when discounted, establishing that the problem is pricing policy rather than the product category itself.

Note: The ::numeric cast is required in PostgreSQL because the ROUND function does not accept double precision values directly.

04 Which are the ten worst-performing products by total profit, including profit per unit and average discount?

Result Cubify CubeX 3D Printer (Double Head): -\$8,880 total | -\$987 per unit | 53% avg discount

```
SELECT
    product_name,
    product_category,
    quantity,
    SUM(order_profits) AS total_profit,
    SUM(order_sales) AS total_sales,
    SUM(order_profits) / SUM(order_sales) AS profit_margin,
    AVG(order_discount) AS avg_discount
FROM
    order_details JOIN product ON order_details.product_id = product.product_id
GROUP BY
    product.product_id, quantity
```

```
ORDER BY
  total_profit ASC, avg_discount DESC
LIMIT 10;
```

What it does: Returns the ten products with the lowest total profit across the dataset, alongside their profit margin and average discount. Including quantity in the GROUP BY allows per-unit context to be surfaced alongside totals. The query identifies two distinct types of loss-maker: products losing money primarily because of extreme discounting, and products that appear structurally unprofitable regardless of how they are priced.

05 Which is the single most profitable product overall, and what is its average discount?

Result Canon imageCLASS 2200 Advanced Copier: +\$25,200 total profit | 12% average discount

```
SELECT
  product_name,
  product_subcategory,
  SUM(order_profits) AS total_profit,
  SUM(order_profits) / SUM(order_sales) AS profit_margin
FROM
  order_details JOIN product ON order_details.product_id = product.product_id
GROUP BY
  product.product_id
ORDER BY
  total_profit DESC
LIMIT 1;
```

What it does: Returns the single product with the highest total profit across the full dataset. Used as a benchmark to illustrate what a high-performing product looks like in terms of category, price point and discount level. The Canon imageCLASS 2200 Advanced Copier generates more profit on its own than the entire Furniture category combined, and does so at an average discount of 12%, well below the 20% threshold identified as the tipping point.

Custom queries written to support Project Unicorn executive summary | Unicorn Database 2015 to 2018